

DEV2 – JAVL – Laboratoire Java

TD 04 – Scrabble

Un mini programme orienté objet

Résumé

Dans ce TD, nous allons coder une version mini-mini du jeu de société *Scrabble*.

Ce TD a pour objectif de développer un premier tout petit programme orienté objet ; quelques objets et une classe principale mettant le tout ensemble. De nouveaux concepts sont abordés ici qui seront affinés plus tard : l'utilisation de *packages*, le mot clé `private`, les tableaux à deux dimensions et les énumérations.

Table des matières

1	Les premiers pas	1
1.1	Une classe <code>Letter</code>	2
1.2	Une classe <code>Board</code>	3
1.3	Une classe <code>Bag</code>	5
1.4	La classe principale <code>App</code>	6
2	Compter les points	9

1 Les premiers pas

Cette première partie sera assez dirigée afin de vous mettre à l'aise avec l'exercice.

Nous allons nous contenter de trois objets pour cet exercice : une classe représentant une lettre (*letter*) du jeu, une classe représentant le plateau de jeu (*board*) et une classe représentant le sac (*bag*) contenant toutes les lettres.

Préalables

Avant de commencer, synchronisez votre dépôt `git` si vous en utilisez un et créez un nouveau projet (via IntelliJ) pour cet exercice.

- ✍ Dans ce nouveau projet, créez un nouveau *package* nommé `g12345.scrabble` et dans ce package, ajoutez une classe `App` contenant une méthode principale (`main`) affichant `SCRABBLE`.

Package Toutes les classes de l'API Java sont regroupées logiquement en *packages*. Un package donne un nom complet à une classe, un nom qualifié. Par exemple :

- ▷ mypackage.MyClass
- ▷ g12345.scrabble.App
- ▷ java.util.ArrayList

L'utilité des packages est qu'ils permettent de regrouper des classes liées sémantiquement entre elles et leur donnent un nom unique.

Pour utiliser la classe `java.util.Scanner` (cfr. infra), il est possible :

- ▷ d'utiliser son nom complet / qualifié ou

```
java.util.Scanner keyboard = new java.util.Scanner(...);
```

- ▷ d'utiliser `import` et le nom court de la classe

```
import java.util.Scanner;
// [cut]
Scanner keyboard = new Scanner(...);
```

C'est la deuxième manière de faire qui sera privilégiée.

Pour préciser qu'une classe fait partie d'un package, on utilise le mot clé `package` comme *première ligne* de la classe.

```
package g12345.scrabble;

public class App {
    // insert code here
}
```

Un nom de package s'écrit en minuscules et correspond à un nom de domaine inversé. Pour nous ce devrait être `be.he2b.esi.g12345.<mon projet>...` mais l'on se contente de `g12345.scrabble`.

1.1 Une classe `Letter`

Pour commencer, notre objet lettre sera très très simple et n'aura qu'un seul attribut¹ : la lettre qu'il représente et qui sera de type `char`.

Étape par étape :

- ✍ Dans votre projet sous IntelliJ, créez une nouvelle classe `Letter` dans le package `g12345.scrabble`.
- ✍ Ajoutez un attribut de type `char` nommé `letter`. Cet attribut sera privé².
- ✍ Ajoutez un constructeur avec un paramètre dont la forme est la suivante :

```
public Letter(char letter) {
    // write something
}
```

- ✍ Ajoutez un *getter* retournant la lettre (l'attribut de type `char`)
- ✍ Dans la méthode `main` de votre classe `App`, déclarez une variable de type `Letter`, instanciez-la et affichez son caractère. Votre code devrait être :

```
Letter l = new Letter('A');
System.out.println(l.getLetter());
```

1. Est-ce que ça a du sens d'avoir un objet avec un seul attribut ? Ben, non. Pas vraiment... mais attend. Lis jusqu'au bout ;-)

2. Pour rendre un attribut privé, il suffit de le faire précéder du mot clé `private`

1.2 Une classe Board

Le plateau de jeu est un peu plus complexe. Il est constitué de 15x15 cases sur lesquelles l'on pose les lettres. Cet objet a donc besoin d'un conteneur pour ces 225 lettres !

En Java, il est possible d'utiliser une liste de listes ou un tableau à deux dimensions. Votre attribut serait donc au choix :

```
import java.util.List;
// [cut]
private List<List<Letter>>> squares;
```

OU

```
private Letter[][] squares;
```

Nous vous proposons d'utiliser un tableau à deux dimensions. En Java, un tableau a une taille fixe dès qu'il est créé. Pour créer un tableau de 15 lignes de chacune 15 colonnes, on peut faire :

```
Letter[][] squares = new Letter[15][15];
```

Pour accéder aux éléments, cela se fait avec la même notation entre crochets ([]).

```
squares[0][0] = new Letter('A');
```

Étape par étape :

- ✍ Créez une nouvelle classe **Board** dans le package **g12345.scrabble**.
- ✍ Ajoutez un attribut **squares** représentant toutes les cases du plateau de jeu.

Attributs privés

Prenons l'habitude de déclarer les attributs *privés*. Un attribut privé peut être accédé au sein de la classe... et c'est tout. Il n'est pas accessible en dehors de celle-ci.

C'est une bonne pratique.

Pour rendre un attribut privé, il suffit de le faire précéder du mot clé **private**.

- ✍ Ajoutez un constructeur qui ne reçoit pas de paramètre et qui crée le plateau de jeu ; il crée le tableau à deux dimensions de lettres (**Letter**).
- ✍ Ajoutez une méthode **get** qui retourne une lettre à une certaine position sur le plateau de jeu.

```
public Letter get(int row, int col){
    // insert code
}
```

Remarque : c'est mieux si le code vérifie que les paramètres **row** et **col** ont une valeur comprises entre 0 et 15 (non compris).

Que faire si les paramètres que l'on reçoit ne conviennent pas ?

Une bonne pratique est de lever une exception (cfr TD 3).

Ici comme l'argument reçu en paramètre ne convient pas s'il n'est pas compris entre 0 et 15, c'est une **IllegalArgumentException** qu'il faudra lever.

- ✎ Ajoutez une méthode qui place une série de lettres dans l'ordre sur le plateau de jeu.

```
public void setLetters(int row, int col, Direction d, Letter[] letters){  
    // insert code  
}
```

- ▷ `row` et `col` représentent la position à laquelle la première lettre sera placée. Les autres suivront.
- ▷ `Direction...` est inconnu !

Créez une nouvelle classe qui s'appelle `Direction` et qui sera une énumération (**enum**). Elle aura deux valeurs : `HORIZONTAL` et `VERTICAL`.

Énumération

Une énumération est une classe particulière qui ne peut prendre que quelques valeurs. Ces valeurs sont fixées lors de l'écriture de la classe.

Une énumération très simple s'écrit par exemple :

```
public enum Direction {  
    HORIZONTAL, VERTICAL;  
}
```

Utilisez cette énumération se fait comme suit :

```
Direction d = Direction.HORIZONTAL;  
Direction d2 = new Direction(); // ERROR
```

- ▷ `Letter[]` attend la liste des lettres à placer sur le plateau.

Attention, ne placez les lettres sur le plateau de jeu que si les cases sont libres³... et que ça ne « déborde » pas du plateau.

Placement sur le plateau

Pour placer une lettre sur le plateau de jeu en utilisant le tableau de lettres reçu en paramètre, il suffit d'écrire :

```
square[row][col] = letters[0];
```

- ▷ c'est mieux de vérifier que `row` et `col` sont bien dans le plateau
- ▷ cet extrait place la première lettre reçue. Il y en a sans doute d'autres.

Pour vérifier qu'une case est libre, il suffit de vérifier si la valeur qui s'y trouve vaut `null`. La valeur `null` signifie l'absence de valeur et donc qu'il n'y a aucune lettre à cet endroit.

- ✎ Dans la méthode `main` de votre classe `App`, déclarez une variable de type `Board` et testez ses méthodes.

Votre test pourrait avoir cette allure :

```
Board board = new Board();  
Letter[] letters = new Letter[] {new Letter('A'), new Letter('B')};  
board.setLetters(0, 0, Direction.HORIZONTAL, letters);
```

3. Pour commencer, ne prévoyez pas les cas où le mot croise l'autre mot « au milieu ». Considérez que l'on place les lettres dans l'ordre « en une fois ».

```
System.out.println(board.get(3,4)); // null
System.out.println(board.get(0,1).getLetter()); // B
```

ArrayIndexOutOfBoundsException

Lorsqu'une instruction du code tente d'accéder à un élément en *dehors* d'un tableau, une `ArrayIndexOutOfBoundsException` est lancée.

Par exemple, avec un tableau de 3 entiers comme ci-dessous, la tentative d'accès à l'élément d'indice 3 — le quatrième élément — qui n'existe pas lancera une telle exception.

```
int[] a = \{3, -14, 15\};
int i = a[3]; // ERROR
```

1.3 Une classe Bag

Au Scrabble toutes les lettres de l'alphabet sont représentées et certaines le sont en plusieurs exemplaires. Par exemple, il y a 15 *E* mais 1 seul *Z*.

La classe « sac de lettres » (*bag*) contiendra toutes les pièces du jeu et permettra de tirer des lettres au hasard pour remplir le chevalet du joueur ou de la joueuse.

Cette classe aura donc comme attribut une collection pouvant contenir en ensemble de lettres. Utilisons une *liste*. En Java, déclarer une liste et l'instancier se fait comme suit :

```
List<Lettre> letters = new ArrayList<>();
```

▷ ce code nécessite deux *import* parce que ces classes sont définies « ailleurs »⁴

```
import java.util.List;
import java.util.ArrayList;
```

Étape par étape :

✍ Créez une nouvelle classe `Bag`.

✍ Ajoutez un attribut `letters` qui contiendra toutes les lettres disponibles.

Cet attribut sera de type `List<Lettre>`.

✍ Ajoutez un constructeur qui ne reçoit pas de paramètre et qui crée toutes les lettres (`Lettre`) et les ajoute à la liste.

Ajouter un élément à une liste se fait avec `add` comme ceci :

```
letters.add(new Lettre('A'));
```

✍ Ajoutez deux méthodes permettant de tirer des lettres au hasard⁵ ; la première tire une seule lettre et la seconde en tire *n*.

```
public Lettre draw(){
    // write something
    return // a letter
}

public List<Lettre> draw(int n){
```

4. Ce genre de classes sont fournies avec l'API du langage. Elles « viennent » avec le langage lorsque l'on installe un JDK. Il faut cependant signaler à Java qu'elles ne se trouvent pas dans le même projet et qu'il doit donc « aller les chercher ».

5. Tirer une lettre au hasard, la retire du sac.

```

// write something else
return // list of letters
}

```

Mélanger

L'API Java propose une méthode pour mélanger des éléments dans une liste.

Pour mélanger la liste de lettres, il suffit d'écrire : `Collections.shuffle(letters)` où `letters` est la liste de lettres. Cette classe se trouve aussi « ailleurs » et nécessite donc un *import*.

Lequel ?

Mon IDE peut-il m'aider ?

1.4 La classe principale App

Reprenez votre classe `App` contenant votre méthode `main`. Nous allons lui ajouter la mécanique du jeu. Il sera nécessaire de :

- ▷ afficher le plateau de jeu ;
- ▷ demander au joueur quelles lettres il veut placer sur le plateau de jeu ;
- ▷ compléter le chevalet du ou de la joueuse et l'afficher ;
- ▷ recommencer...

Commençons par afficher les éléments et ensuite, nous ferons les lectures des entrées des utilisatrices.

Étape par étape :

- ✍ Ajoutez deux méthodes pour l'affichage du plateau de jeu (un `Board`) et le chevalet du ou de la joueuse (une liste de lettres).

- ▷ la première a l'allure suivante de la figure 2
- ▷ la seconde, nous vous laissons l'écrire

```

private static void display(List<Letter> rack){
    // write something
}

```

- ✍ Ajoutez une méthode `run` (ou d'un autre nom) qui contiendra la *boucle de jeu*. Cette boucle se répète tant que l'on ne quitte pas le jeu et fait : afficher le plateau, afficher le chevalet, demander au joueur ou à la joueuse ce qu'elle veut faire, placer les lettres, recharger le chevalet... et recommencer.

Il y a deux actions : quitter le jeu ou placer des lettres à un endroit du plateau.

- ▷ Pour quitter le jeu, il suffit d'entrer la commande (une chaîne de caractères) `quit`.

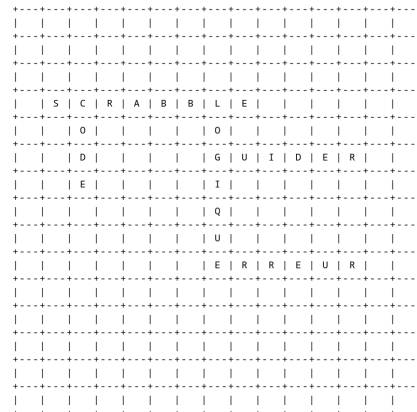


FIGURE 1 – Exemple de *board*

```
private static void display(Board board) {
    for (int i = 0; i < 15; i++) {
        System.out.println("+---".repeat(15) + "+");
        for (int j = 0; j < 15; j++) {
            Letter letter = board.get(i, j);
            if (letter != null) {
                System.out.print("| " + letter.getLetter() + " ");
            } else {
                System.out.print("|  ");
            }
        }
        System.out.println("|");
    }
    System.out.println("+---".repeat(15) + "+");
}
```

FIGURE 2 – Exemple de code de la méthode `display`

- ▷ Pour placer des lettres, il faut entrer une commande de la forme
`set <row> <col> <direction> <indice lettre> [indice lettre...]`.

Par exemple : `set 7 3 h 0 1` qui place horizontalement à partir de la ligne 7 et de la colonne 3 les lettres d'indice 0 et 1 dans le chevalet du ou de la joueuse.

Comment lire une commande de l'utilisateurice ?

L'accès au clavier de la machine, se fait grâce à une classe particulière : la classe `System`. L'entrée standard — le clavier — s'appelle `in`.

La lecture de cette entrée, se fait quand-à-elle grâce à la classe `Scanner` qui nous permettra de lire directement une ligne entrée au clavier. La méthode `readLine`.

Une manière de lire une ligne est de définir une variable constante — grâce au mot clé `final` — représentant le clavier et d'appeler la méthode `readLine`. La chaîne de caractère reçue pourra alors être traitée.

```
final Scanner IN = new Scanner(System.in);
// ...
String in = IN.nextLine();
```

Traiter la chaîne de caractère c'est la découper en sous-chaînes afin de séparer chaque mot.

Avec la chaîne `in` obtenue ci-dessus, la découpe en mots se fait par :

```
String[] elements = in.split("\\s+");
```

- ▷ Nous pourrions écrire `in.split(" ")` qui couperait la chaîne à chaque espace blanc mais le code se comporterait mal avec 2 espaces blancs ou une tabulation ou...

Utiliser `\\s+` veut dire que l'on découpe sur 1 ou plusieurs « espaces blancs » au sens large. C'est-à-dire les simples espaces mais également les tabulations. C'est plus robuste.

- ▷ `elements[0]` contiendra alors `quit` ou `set` ou une autre valeur qui ne nous intéresse pas et dans ce cas, il faudra dire que c'est une erreur.

Vérifier l'égalité de deux chaînes se fait à l'aide de la méthode `equals` comme ceci :

```
String command = elements[0];
if (command.equals("quit")) {
    // exit program
} else if (command.equals("set")) {
    // do something
} else {
    // bad command
}
```

- ▷ Certaines parties de la commande devront être converties du type chaîne (`String`) en entiers (`int`) pour pouvoir servir d'indice dans le jeu.

Convertir une chaîne en entier se fait grâce à :

```
int i = Integer.parseInt(s);
```

- ▷ où l'on suppose que `s` est une chaîne. Par exemple `elements[1]`.
- ▷ si la conversion ne peut se faire, une exception sera lancée (cfr. TD3)

À ce stade le jeu devrait être fonctionnel. Chez moi la méthode `main` à cette allure :

```
public static void main( String[] args ) {
    System.out.println( "SCRABBLE" );
    Bag bag = new Bag();
    Board board = new Board();
    run(bag, board);
}
```

Ce que le programme ne fait pas

Le programme en l'état a « quelques limites » :

1. Il ne permet pas que le mot ajouté soit croisé à un autre endroit qu'à la première lettre.
2. Il ne vérifie pas que le mot placé... est un mot qui existe dans le dictionnaire. Pour l'instant, c'est un ensemble de lettres.
3. Si je me trompe, je me trompe. Il n'y a pas de moyen de revenir en arrière.
4. ...

Malgré ces limites. Nous avons quelque chose de fonctionnel et nous pouvons jouer un peu.

2 Compter les points

Cette partie est facultative. Si vous n'avez pas le temps de la faire, passez la et revenez-y plus tard (ou pas).

Dans cette deuxième partie, nous allons être moins directif et vous laisser faire un peu plus tout seul ou toute seule.

Au Scrabble, les lettres ont une valeur. Par exemple le *E* vaut 1 point tandis que le *Z* en vaut 10.

En détail :

- ▷ 0 point : Joker⁶ ×2
- ▷ 1 point : E ×15, A ×9, I ×8, N ×6, O ×6, R ×6, S ×6, T ×6, U ×6, L ×5
- ▷ 2 points : D ×3, G ×2, M ×3
- ▷ 3 points : B ×2, C ×2, P ×2
- ▷ 4 points : F ×2, H ×2, V ×2
- ▷ 8 points : J ×1, Q ×1
- ▷ 10 points : K ×1, W ×1, X ×1, Y ×1, Z ×1

Source [Wikipedia](#)

Ajoutez la notion de points et de score à votre programme.

Étape par étape :

- ✍ Dans la classe **Letter**, il faudra ajouter un attribut représentant la valeur de la lettre.
- ✍ La méthode **setLetters** de la classe **Board** pourrait retourner une valeur : le score des lettres placées.
- ✍ Ce score est maintenu dans la classe principale et affiché à chaque tour.

À vous de jouer...

6. Nous n'avons à priori pas utilisé les Joker mais vous pouvez les ajouter si vous voulez.